

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

February 03, 2026
Tuesday



Age prediction from DNA methylation



Input: $\mathbb{X}$
“CpG sites”

Sample	CpG ₁	CpG ₂	CpG ₃
1	0.20	0.35	0.60
2	0.30	0.45	0.65
3	0.40	0.55	0.70
4	0.50	0.65	0.75

Regression

Target: $\mathbb{Y}$
“Age”

→ Function: f →

$f(\mathbb{X}) \rightarrow \mathbb{Y}$

Age
25
35
45
55

$\mathbb{X} \in \mathbb{R}$

$\mathbb{Y} \in \mathbb{Z}$
(Numerical output)

Gradient Descent of MSE (1 sample)

$$\Delta w = -\alpha \cdot \frac{\partial L}{\partial w}$$

$$L = (y - \hat{y})^2$$

$$= (y - f(x))^2$$

$$= y^2 + f(x)^2 - 2yf(x)$$

$$= y^2 + (wx + b)^2 - 2y(wx + b)$$

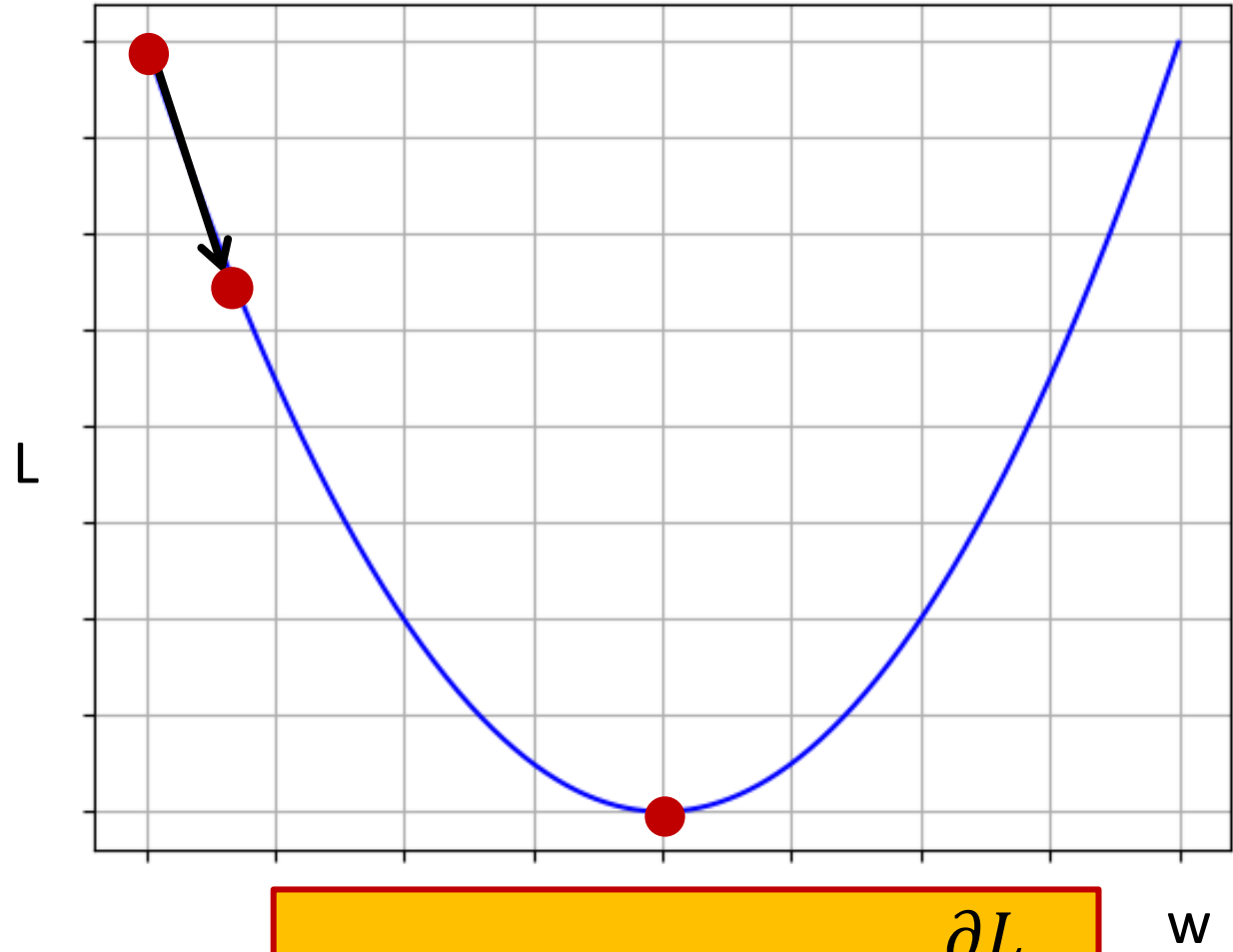
$$= y^2 + w^2x^2 + b^2 + 2wxb - 2ywx - 2yb$$

$$\frac{\partial L}{\partial w} = ?$$

$$\frac{\partial L}{\partial w} = 2wx^2 + 2xb - 2yx$$

$$\frac{\partial L}{\partial w} = 2x(wx + b - y)$$

$$\frac{\partial L}{\partial w} = 2x(\text{error})$$



$$w_{new} = w_{old} - \alpha \cdot \frac{\partial L}{\partial w}$$

Linear regression using Matrix Operations

```
def compute_gradients(X, y, y_pred):  
    n = len(y)  
  
    error = y_pred - y  
  
    dw = (2 / n) * X.T @ error  
    db = (2 / n) * np.sum(error)  
  
    return dw, db
```

$$\text{error} = \begin{bmatrix} \hat{y}_1 - y_1 \\ \hat{y}_2 - y_2 \\ \hat{y}_3 - y_3 \\ \hat{y}_4 - y_4 \end{bmatrix}$$

$$X^T = \begin{bmatrix} x_{11} & x_{21} & x_{31} & x_{41} \\ x_{12} & x_{22} & x_{32} & x_{42} \\ x_{13} & x_{23} & x_{33} & x_{43} \end{bmatrix}$$

$$\begin{bmatrix} \sum_i x_{i1}(\hat{y}_i - y_i) \\ \sum_i x_{i2}(\hat{y}_i - y_i) \\ \sum_i x_{i3}(\hat{y}_i - y_i) \end{bmatrix}$$

Today's goal - Learn about practical considerations when doing ML

(1) Model fit and Cross-validation

(2) Data pre-processing

(3) Regularization

(4) Class Activity: ML in practice using ScikitLearn

(5) Evaluation

Model fitting and Cross-validation

Learning function f



Input: $\mathbb{X}$
“CpG site 1”

Regression

Target: $\mathbb{Y}$
“Age”

Linear function

$\mathbb{X} \in \mathbb{R}$

$x^{(1)}$ 0.20

$x^{(2)}$ 0.30

$x^{(3)}$ 0.40

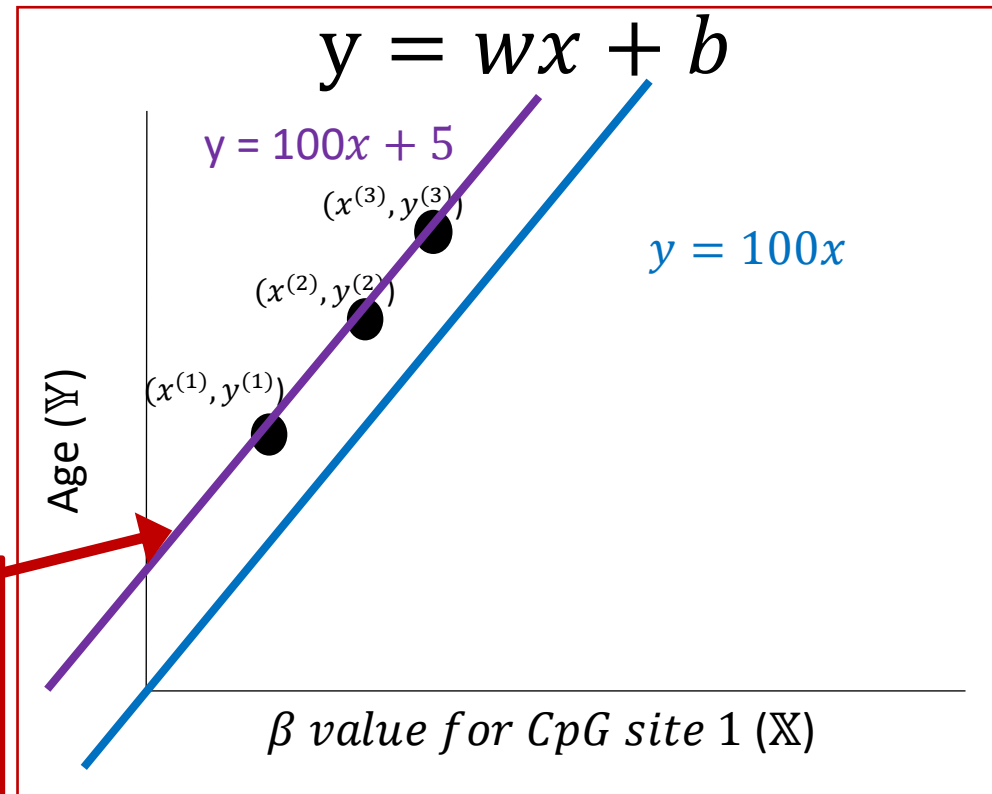
$y^{(1)}$ 25

$y^{(2)}$ 35

$y^{(3)}$ 45

$\mathbb{Y} \in \mathbb{Z}$

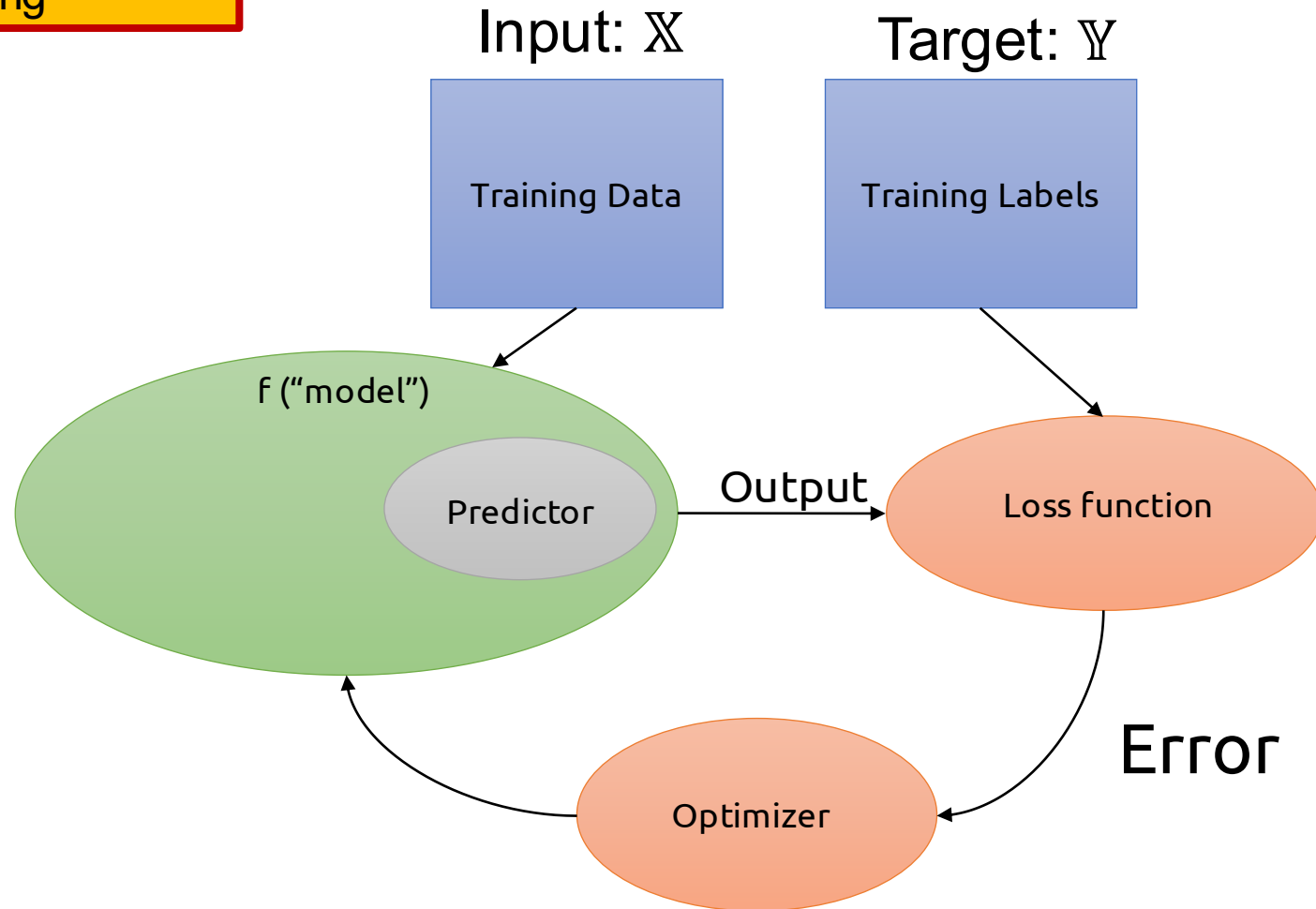
(Numerical output)



Only the
line with
bias can fit
the data

“Classic” Supervised Learning in Machine Learning

Training



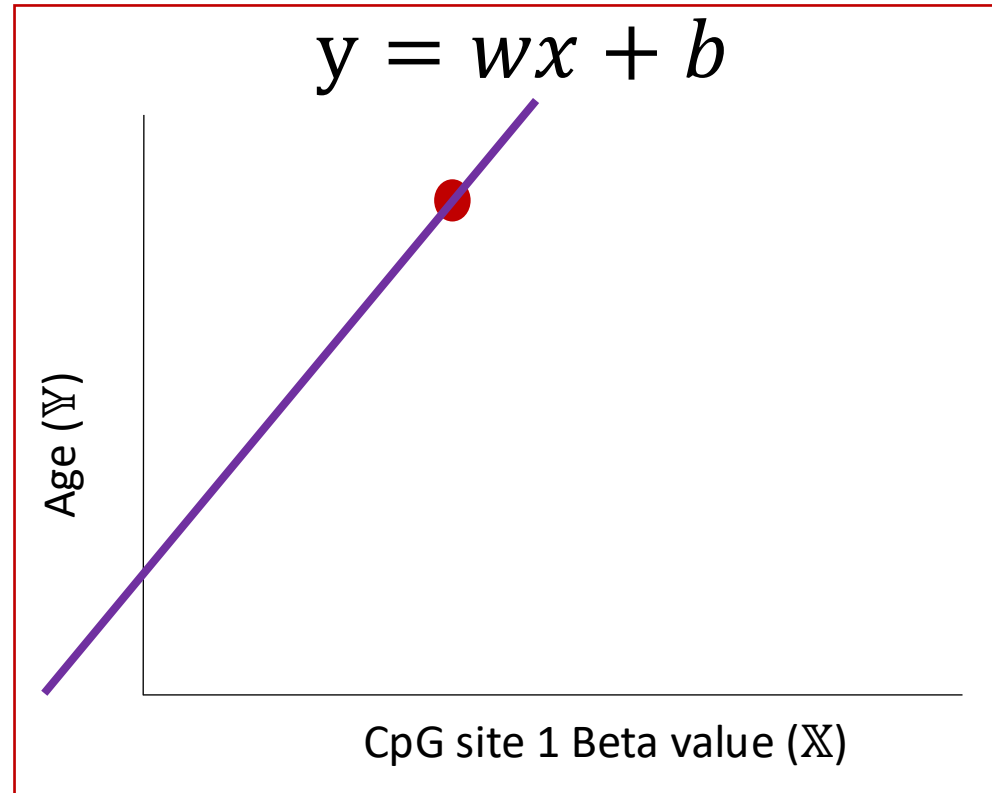
Testing our model



“CpG site 1”

$$x' = 0.6$$

Linear function



“Age”

Prediction

$$y' = 65$$

Testing our model



“CpG site 1”

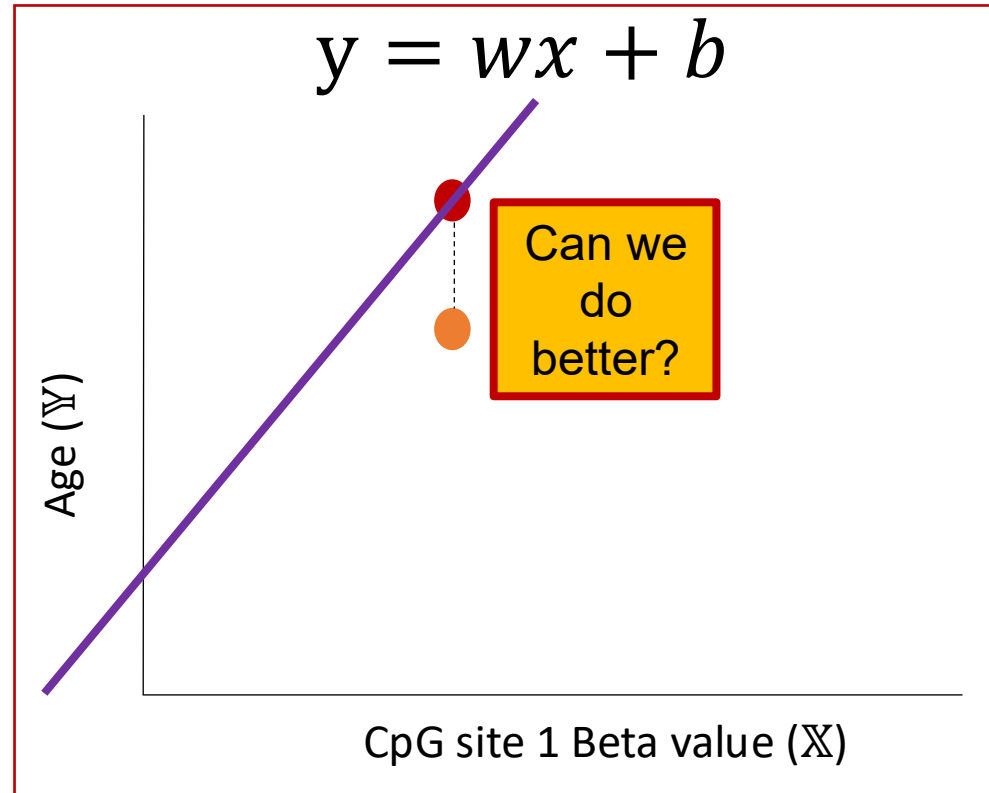
$$x' = 0.6$$

$$\hat{x} = 0.6$$

Real-world deployment

Linear function

$$y = wx + b$$



“Age”

Prediction

$$y' = 65$$

True observation

$$\hat{y} = 55$$

(An outlier is a data point that differs significantly from other observations)

(Image only for explaining concept, not drawn accurately)

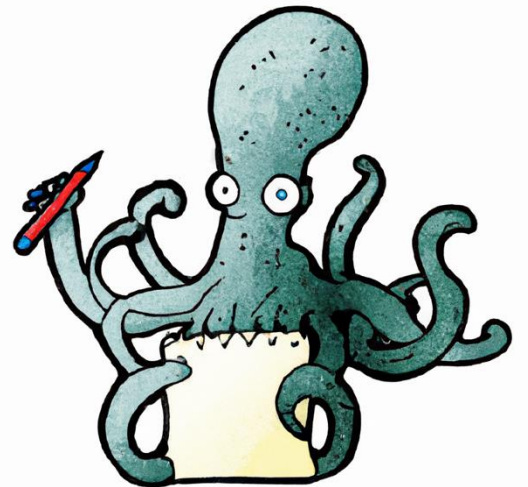
Can we do better? – May be

How?

Option 1: Collect more data and retrain

Option 2: Try a different function

Option 3: Do both 1 and 2



Learning better models – Collect more data



“CpG site 1”

$$x^{(1)} = 0.2$$

$$x^{(2)} = 0.3$$

$$x^{(3)} = 0.4$$

.

.

.

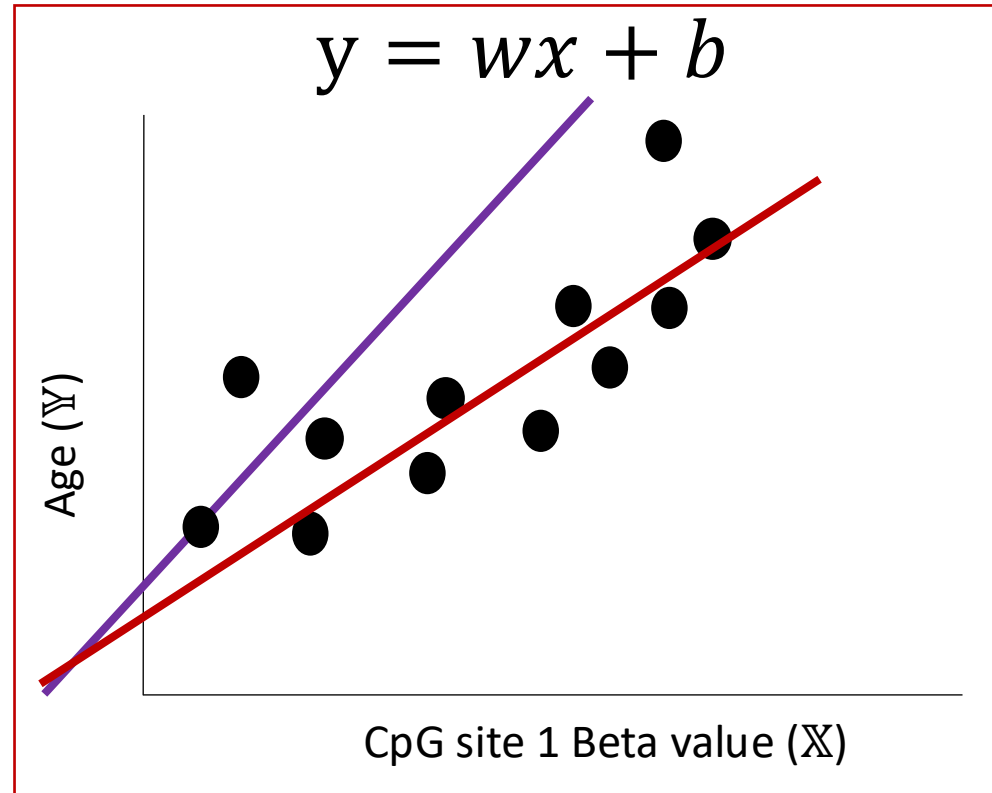
.

$$x^N = \dots$$

$$\mathbb{X} \in \mathbb{R}$$

Linear function

$$y = wx + b$$



“Age”

$$y^{(1)} = 25$$

$$y^{(2)} = 35$$

$$y^{(3)} = 40$$

.

.

.

.

$$y^N = \dots$$

$$\mathbb{Y} \in \mathbb{R}$$

(Numerical output)

Learning better models – Try different functions



“CpG site 1”

$$x^{(1)} = 0.2$$

$$x^{(2)} = 0.3$$

$$x^{(3)} = 0.4$$

⋮

⋮

⋮

⋮

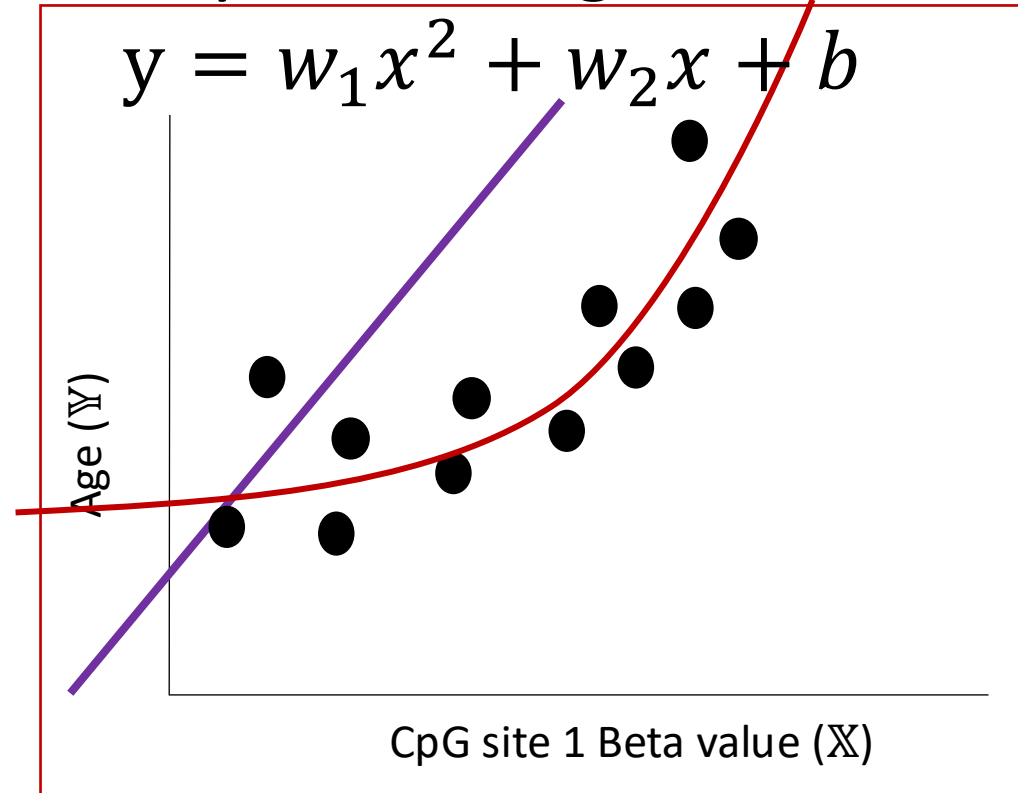
$$x^N = \dots$$

$$\mathbb{X} \in \mathbb{R}$$

Non-linear function

Polynomial regression

$$y = w_1 x^2 + w_2 x + b$$



“Age”

$$y^{(1)} = 25$$

$$y^{(2)} = 35$$

$$y^{(3)} = 40$$

⋮

⋮

⋮

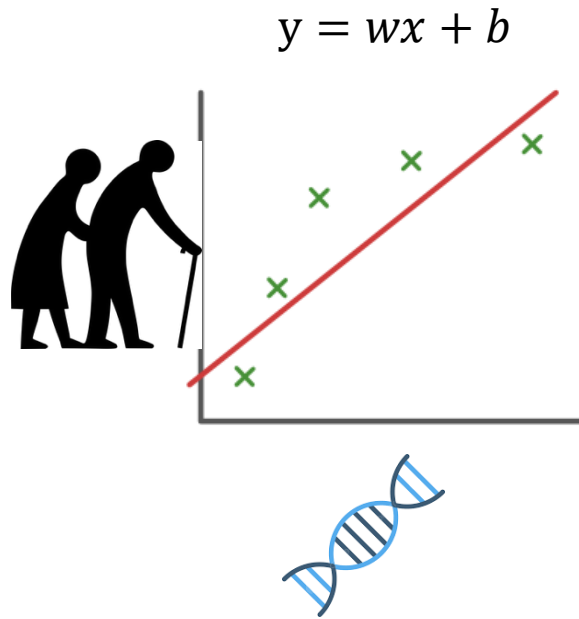
⋮

$$y^N = \dots$$

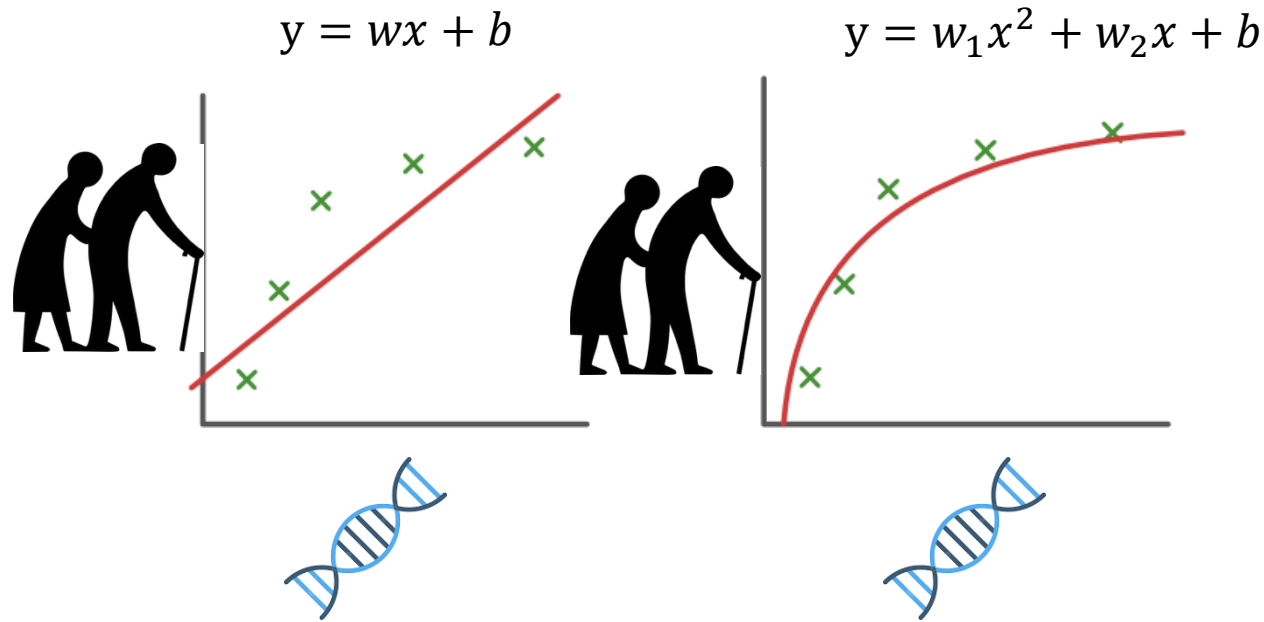
$$\mathbb{Y} \in \mathbb{R}$$

(Numerical output)

How to know which function is the best?



How to know which function is the best?

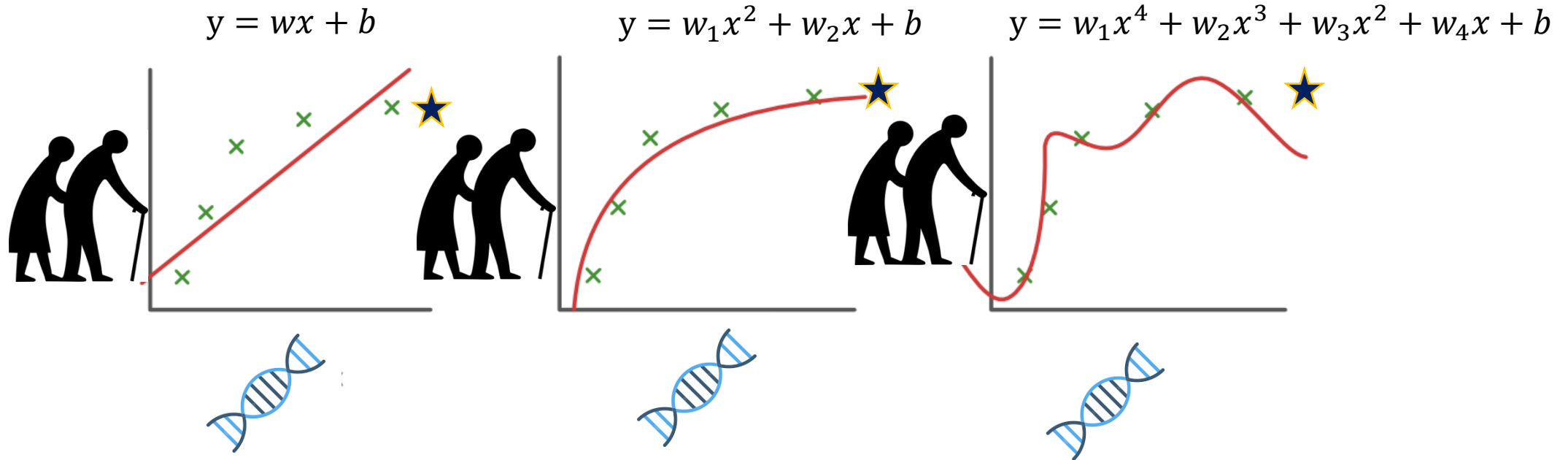


How to know which function is the best?

Underfit

Good fit

Overfit



“My model is not doing that well on the given data and new data” ☹

“My model is doing well on the given data AND the new data point!! ☺

“My model is doing really well on the given data!! ☺

“The performance is bad on new data point” ☹

How to train your ~~dragon~~ model

Input: $\mathbb{X}$
“CpG site 1”

Target: $\mathbb{Y}$
“Age”

Training set	$x^{(1)} = 0.2$	$y^{(1)} = 25$
	$x^{(2)} = 0.3$	$y^{(2)} = 35$
	$x^{(3)} = 0.4$	$y^{(3)} = 40$
	.	.
Validation set	.	.
	.	.
	.	.
	.	.
Test set	.	.
	$x^N = \dots$	$y^N = \dots$

How can we use the data we have to test for the fit?

Training phase

“My model is not doing that well on the training and validation data” ☹

Age ($\mathbb{Y}$)

“My model is doing well on the training data!! ☺

CpG site 1 Beta value ($\mathbb{X}$)

“The performance is bad on validation data” ☹

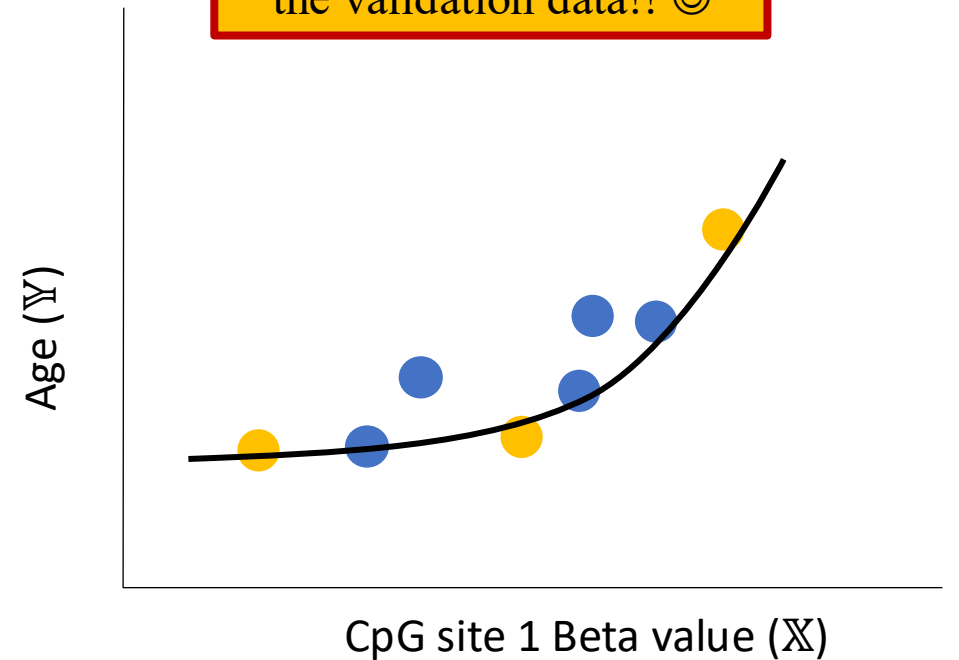
How to train your ~~dragon~~ model

	Input: $\mathbb{X}$ "CpG site 1"	Target: $\mathbb{Y}$ "Age"
Training set	$x^{(1)} = 0.2$ $x^{(2)} = 0.3$ $x^{(3)} = 0.4$.	$y^{(1)} = 25$ $y^{(2)} = 35$ $y^{(3)} = 40$.
Validation set	.	.
	.	.
	.	.
	.	.
Test set	.	.
	.	.
	$x^N = \dots$	$y^N = \dots$

Cross-validation (1-fold)

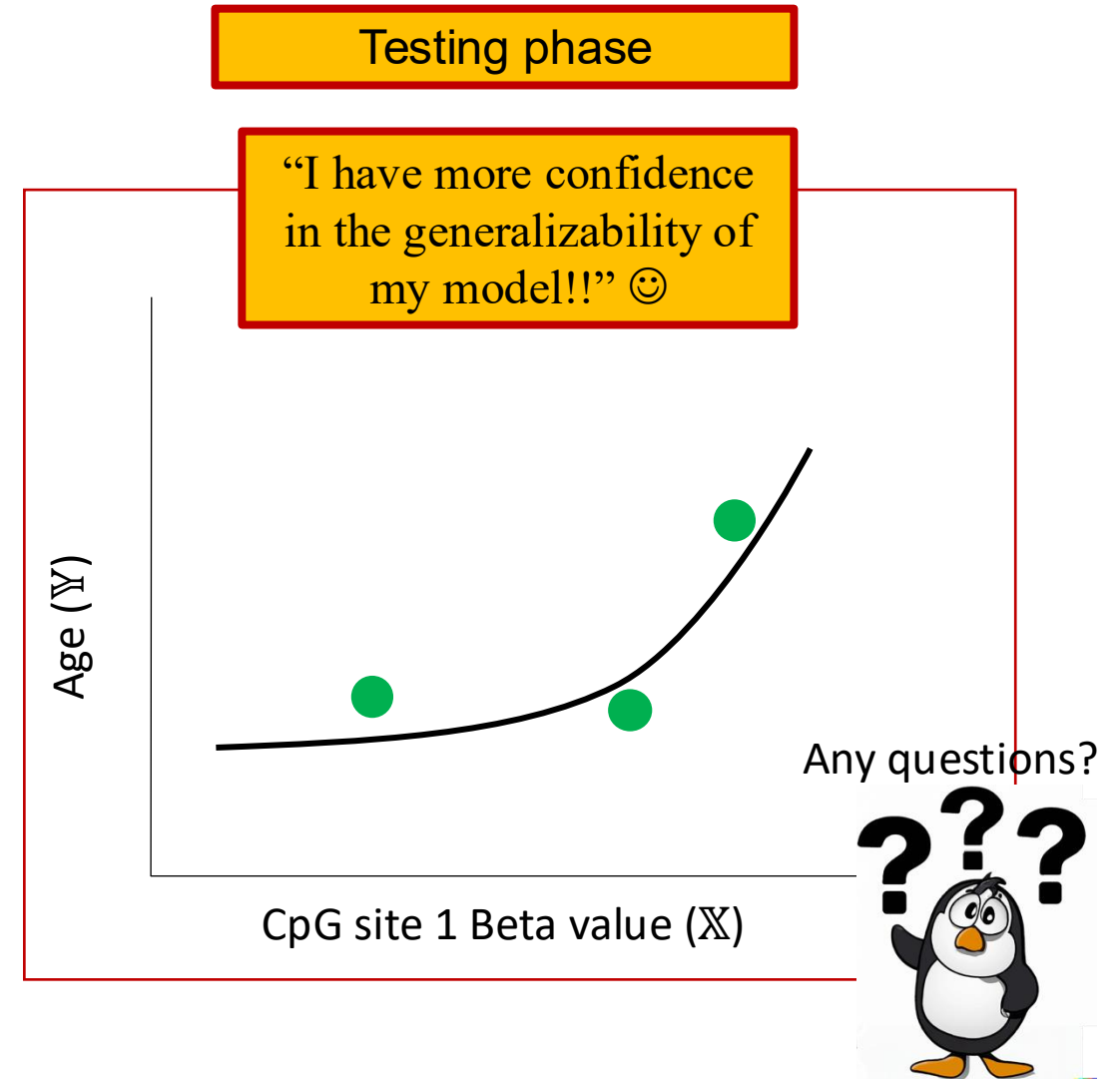
Training phase

"My model is doing well on the training data AND the validation data!! 😊"



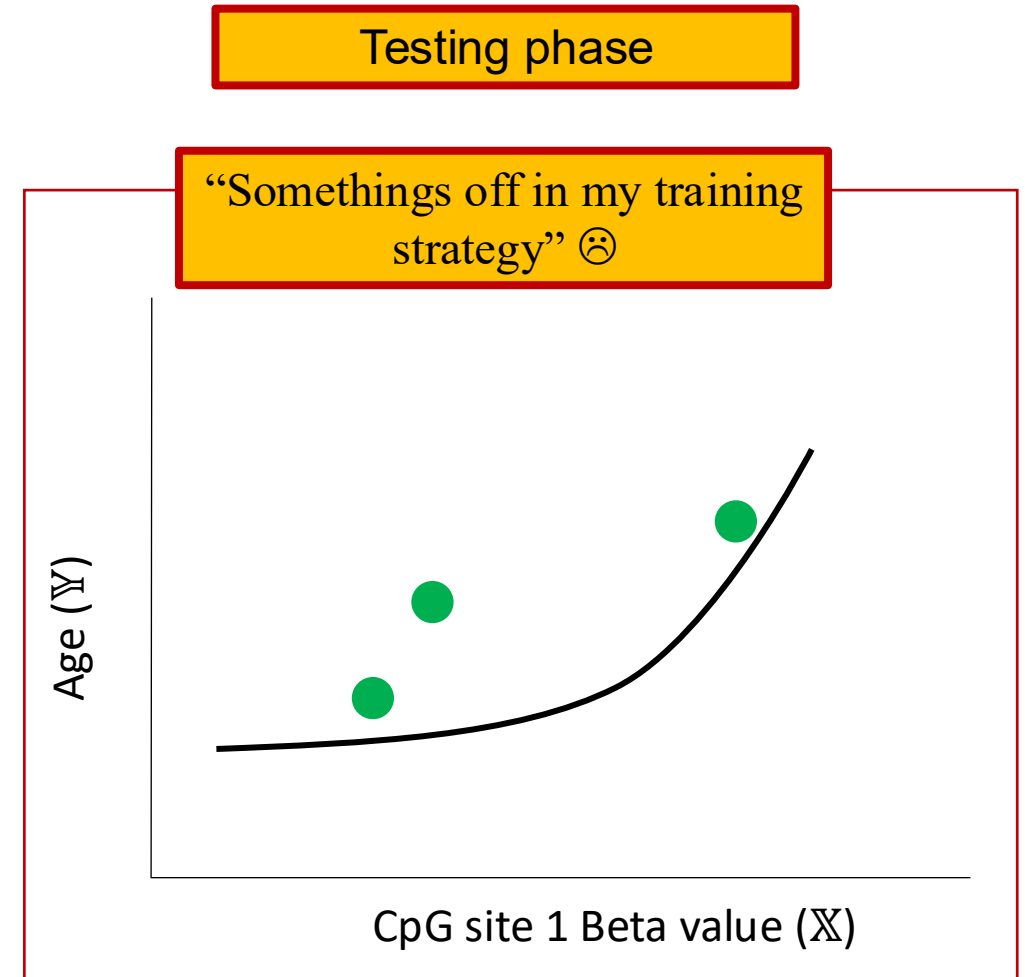
How to train your ~~dragon~~ model

	Input: $\mathbb{X}$ "CpG site 1"	Target: $\mathbb{Y}$ "Age"
Training set	$x^{(1)} = 0.2$ $x^{(2)} = 0.3$ $x^{(3)} = 0.4$.	$y^{(1)} = 25$ $y^{(2)} = 35$ $y^{(3)} = 40$.
Validation set	.	.
Test set	.	.
	$x^N = \dots$	$y^N = \dots$



How to train your ~~dragon~~ model

	Input: $\mathbb{X}$ "CpG site 1"	Target: $\mathbb{Y}$ "Age"
Training set	$x^{(1)} = 0.2$ $x^{(2)} = 0.3$ $x^{(3)} = 0.4$.	$y^{(1)} = 25$ $y^{(2)} = 35$ $y^{(3)} = 40$.
Validation set	.	.
	.	.
	.	.
	.	.
Test set	.	.
	.	.
	$x^N = \dots$	$y^N = \dots$



Real world data tends to be complicated!



Input: $\mathbb{X}$

Target: $\mathbb{Y}$

Sample	CpG ₁	CpG ₂	CpG ₃
1	0.20	0.35	0.60
2	0.30	0.45	0.65
3	0.40	0.55	0.70
4	0.50	0.65	0.75

➡ Function: f ➡

Age
25
35
45
55

Now our function needs to capture the relationships of the combined feature space of the input and the output!

Data preprocessing

Dealing with messy data: preprocessing

- Always necessary (unless it's preprocessed for you...)
- Goal: get messy, unstructured data into **usable** data for your model
 - Ultimately, data must be converted into **matrices (or tensors)** to be fed to your model
- What exactly you need to do is dataset- and context- dependent
 - Handle missingness
 - Quantify non-numerical entities (e.g. categories $\rightarrow$ 1/0)
 - Drop outliers
 - Normalize/standardize inputs

Dealing with messy data: preprocessing

- Always necessary (unless it's preprocessed for you...)
- Goal: get messy, unstructured data into **usable** data for your model
 - Ultimately, data must be converted into **matrices (or tensors)** to be fed to your model
- What exactly you need to do is dataset- and context- dependent
 - **Handle missingness**
 - Quantify non-numerical entities (e.g. categories $\rightarrow$ 1/0)
 - Drop outliers
 - **Normalize/standardize inputs**

Missing Data

Example: Conducting a survey

Gender Race Education Earnings

Group	Gender	Race	Education	Earnings
1	Red	Red	Red	Red
2	Red	Red	Blue	Blue
3	Red	Blue	Red	Blue
4	Blue	Red	Red	Blue
5	Blue	Blue	Red	Red
6	Blue	Blue	Red	Blue
7	Blue	Blue	Blue	Red
8	Blue	Red	Blue	Blue
9	Red	Blue	Blue	Blue
10	Blue	Blue	Blue	Blue

Types of missing data

	Gender	Race	Education	Earnings
•				
•				
•				
•				
•				
•				
•				
•				
•				
•				

- **Missing completely at random:** if the probability of missingness is the same for all units

If the computer had a glitch when recording values

- **Missing at random:** probability a variable is missing depends only on available information

If gender and race are recorded for all the people in the survey, then “education” is missing at random if the probability of nonresponse to this question depends only on these other, fully recorded variables.

- **Missingness that depends on unobserved predictors:** missingness depends on information that has not been recorded and this

Suppose that people with college degrees are less likely to reveal their earnings, having a college degree is predictive of earnings, and there is also some nonresponse to the education question.

Handling Missing Data

- **Missing completely at random:** if the probability of missingness is the same for all units

Do nothing, let algorithm handle N/A values

Discard missing data

- **Missing at random:** probability a variable is missing depends only on available information

Imputation: Mean/Median, Random value, Most frequent value, k-nearest neighbor, Regression

- **Missingness that depends on unobserved predictors:** missingness depends on information that has not been recorded and this information also predicts the missing values

Hard case!

Missing Data issue is common in biology/health domains

- Clinical data: missing timepoints in patient's history
- Incomplete surveys
- Low quality experiment/scan : noisy data
- Missing experiments for certain cell types

Any questions?



Normalization/Standardization

Data Normalization:

It's the process of rescaling data into a standard range, usually between 0 and 1.
This is particularly useful when dealing with features that have varying scales.

Data Standardization:

This technique transforms data to have a mean of 0 and a standard deviation of 1.
Makes it more compatible with algorithms that assume a normal distribution of the data.

Why Normalize Data?

Equalization of Scales: It ensures that no single feature dominates the analysis due to its magnitude.

Convergence: Many machine learning algorithms converge faster when input features are within a specific range.

Interpretability: Normalized data is easier to understand as all values are within a common range.

Normalization/Standardization

	input	standardized	normalized
0	0	-1.46385	0.0
1	1	-0.87831	0.2
2	2	-0.29277	0.4
3	3	0.29277	0.6
4	4	0.87831	0.8
5	5	1.46385	1.0

Normalization/Standardization

$$x_{norm}^{(i)} = \frac{x^{(i)} - \mathbf{X}_{min}}{\mathbf{X}_{max} - \mathbf{X}_{min}}$$

*min-max scaling
("normalization")*

	input	standardized	normalized
0	0	-1.46385	0.0
1	1	-0.87831	0.2
2	2	-0.29277	0.4
3	3	0.29277	0.6
4	4	0.87831	0.8
5	5	1.46385	1.0

Note:

Normalization is **sensitive to outliers**. One extreme value can squash everything else.

Normalization/Standardization

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

standardization

$$x_{norm}^{(i)} = \frac{x^{(i)} - \mathbf{X}_{min}}{\mathbf{X}_{max} - \mathbf{X}_{min}}$$

*min-max scaling
("normalization")*

Note:

Standardization keeps the **shape of the distribution**, just re-centered and rescaled.

	input	standardized	normalized
0	0	-1.46385	0.0
1	1	-0.87831	0.2
2	2	-0.29277	0.4
3	3	0.29277	0.6
4	4	0.87831	0.8
5	5	1.46385	1.0

Note:

Normalization is **sensitive to outliers**. One extreme value can squash everything else.

Any questions?



Horvath's clock: Tens of thousands of CpG sites, hundreds of samples



Input: $\mathbb{X}$

Target: $\mathbb{Y}$

Sample	CpG ₁	CpG ₂	CpG ₃
1	0.20	0.35	0.60
2	0.30	0.45	0.65
3	0.40	0.55	0.70
4	0.50	0.65	0.75



Function: f



Age
25
35
45
55

Why is this a problem??

Too many features compared to samples ($p \gg n$)

- **Spurious Correlations:**

With many features and few examples, it is highly likely that some features will correlate perfectly with the target variable purely by random chance

- **Overfitting:**

Models become memorize patterns to fit every noise point in the small training set, which ruins their ability to generalize to new, unseen data

- **Underdetermined Systems:**

In linear regression, having more features than samples creates an "underdetermined" system. This means there are infinite possible solutions (parameter estimates) that can fit the data perfectly, making it impossible to identify the true relationship.

How can you fix this issue?

Regularization

Regularization - why?

This approach leaves the model architecture unchanged, and instead only modifies the ***loss***.

- Adds an additional term to our existing loss function

Remember insight from before:

- Overfitting is correlated with the model relying **too heavily** on **too many *different*** features/correlations

Can we design a loss function that penalizes:

1. Heavy reliance on any correlation in the data?
2. Reliance on too many different correlations in the data?

Regularization - L1 vs L2

L2 regularization

- $\lambda \sum_{j=1}^n |W_j|^2$
- Penalize sum of squared weights
- **Effect:** keeps all weights smooth and small-ish, i.e. model can't learn to rely too heavily on any single pattern in the data

L1 regularization

- $\lambda \sum_{j=1}^n |W_j|$
- Penalize absolute value of weights
- **Effect:** tends to produce *sparse weights* (i.e. many zero-valued weights) → prevents the model from relying on too many different patterns in the data

For both, this is a term added to the existing loss function.

λ controls the strength of the penalty

Regularization - L1 vs L2

Note:

Regularization assumes comparable scales, so standardization is important

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

Lasso Regression

L2 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M W_j^2$$

Ridge Regression

Loss function

Regularization
Term

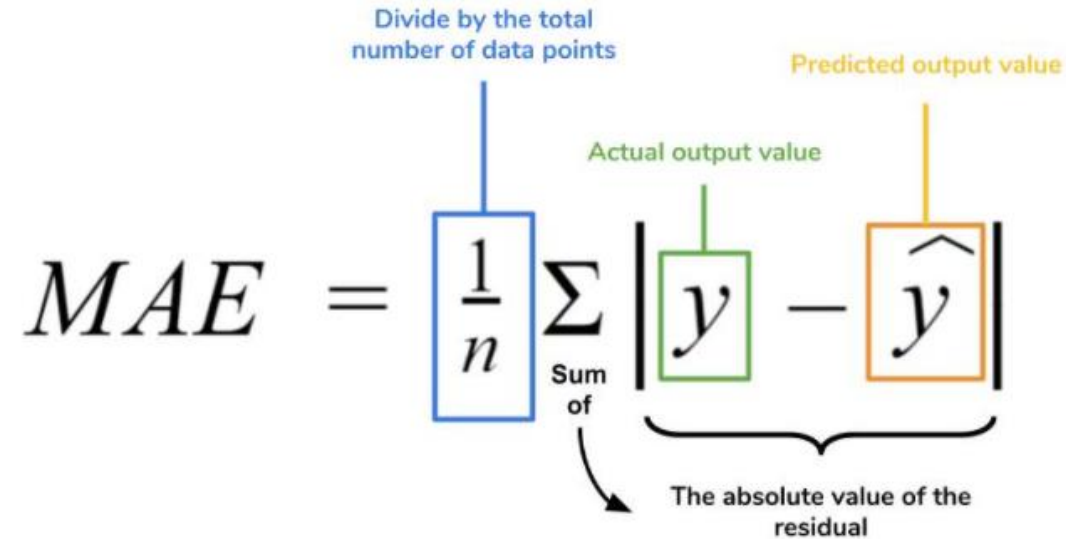
ElasticNet (Horvath's clock)

$$\frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=1}^p x_{ij} W_j \right)^2 + \lambda_1 \sum_{j=1}^p |W_j| + \lambda_2 \sum_{j=1}^p W_j^2$$

Class activity

ML in practice Demo

Evaluating Regression performance



The diagram illustrates the Mean Absolute Error (MAE) formula with color-coded annotations. A blue box around $\frac{1}{n}$ is labeled "Divide by the total number of data points". A green box around y is labeled "Actual output value". An orange box around $\hat{y}$ is labeled "Predicted output value". A bracket under the difference $y - \hat{y}$ is labeled "The absolute value of the residual". The summation symbol Σ is labeled "Sum of".

$$MAE = \frac{1}{n} \sum \left| y - \hat{y} \right|$$

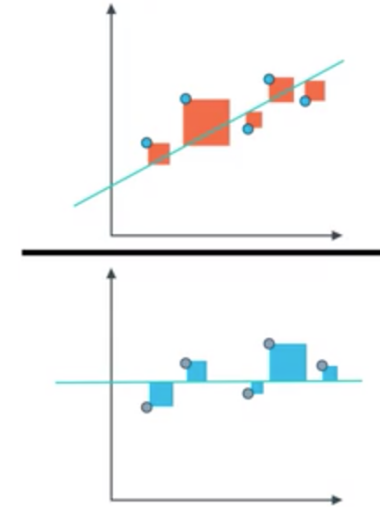
$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Evaluating Regression performance

○ R2 SCORE

- BAD MODEL
The errors should be similar.
R2 score should be close to 0.
- GOOD MODEL
The mean squared error for the linear regression model should be a lot smaller than the mean squared error for the simple model.
R2 score should be close to 1.

$$R^2 = 1 -$$



$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Any questions?



Recap: Learn about practical considerations when doing ML

(1) Model fit and Cross-validation

Homework reading:

[Chapter 5 in CIML book](#)

(2) Data pre-processing

(3) Regularization

(4) Class Activity: ML in practice using ScikitLearn

(5) Evaluation

Wrap up



What was the clearest point today?

What was the muddiest point today?

